

My VIM Cheat Sheet

Plugins

NERDTree

<Leader> t Focus to NERDTree window
<Leader> e Toogle NERDTree window
<C-f> Highlight the current file on NERDTree window

Jedi-vim

<C-Space> Python Auto-Completion
<C-n> / <C-p> Next/Previous suggestions
<Leader>g Goto assignment
<Leader>d Goto definition
<Leader>s Goto (typing) stub
K Show Documentation/Pydoc
<Leader>r Renaming
<Leader>n Show usages
:Pyimport os Opens the 'os' module

Vim-Commentary

gcc Comment a line
gc<motion> Comment the target motion
gc On visual: Comment the selection

Vim-slime

<C-c><C-c> Send text to REPL
<C-c>v Reconfigure target

VIM

<leader> space
.
Repeats previous command

Movement

<C-d> Move half-screen down.
<C-u> Move half-screen up.
<C-f> Move full screen forward.
<C-b> Move full screen backward.
H / M / L Bring cursor to top/middle/bottom of screen.
3H Bring cursor to 3 lines from top of screen.

3L Bring cursor to 3 lines from bottom of screen
} Go forward by paragraph (the next blank line)
{ Go backward by paragraph (the previous blank line)

<C-w> + h/j/k/l or <C-number> Move to * window.
<C-w> + H/J/K/L Move window to * side.
<C-w> + +/- Increase/Decrease window height by 1.
<C-w> - Decrease window height by 1.
<C-w> + >/< Increase/Decrease window width by 1.
<C-w> + > Increase window width by 1.
Ctrl + w + < Decrease window width by 1.
<C-w> + = Equalize height and width of all windows.

Buffers, Windows and Tabs

:ls List all buffers in session.
:bn/:bp Navigate to next/previous buffer.
:bp or :b# Navigate to previous buffer.
:bf/:bl Navigate to first/last buffer.
:bd Delete current buffer.
:bd1 Delete buffer labeled "1".

gt Next tab
gT Previous tab
<C-t> :tabnew
<C-^> Return to previous file
<Leader>wo Open current split in new tab
<C-hjkl> Switch between splits

:vs ~/file Open file in vertical pane.
:sp ~/file Open file in horizontal pane.
:vert sb2 Open buffer 2 in vertical pane.
:sb2 Opens buffer 2 in horizontal pane.

:e ~/file Edit a file.
:tabe ~/file Edit file in new tab.

Editing

J Join lines with comments
w/b Go to next/previous delimiter.
W/B Go to next/previous word.
\$ Go to end of line.
^ Go to first letter of a line.
0 Go to beginning of a line.

gg/G Go to beginning/end of the file.
{/} Go to beginning/end of paragraph.
:8 Go to line 8.

fP/FP Find next/previous P in the line.
tP/TP Find delimiter before/after next P.
;/, Go to next/previous find.

yiw Yank(copy) inside word
viwp Select inside word and paste
viw"Op Select inside word and paste content of register 0
di"/ci" Delete/change text inside "".
da"/ca" Delete/change text inside "", delete "".
u Undo change.
Ctrl + r Redo change.
v Visual mode. j/k to highlight text.
V Visual Line mode. Highlight full lines.

In Visual mode, pressing % under a bracket highlights until matching bracket.
>>/<< Indent/unindent in normal mode.
=3j Autoindent 3 lines after cursor.

Search and Replace

:s/foo/bar/g Replace each foo by bar (current line).
:%s/foo/bar/g Replace each foo by bar (all lines).
:%s/foo/bar/gc Replace each foo by bar (asking).

* Search for word under the cursor.
/[searchterm] Search. n/N for next/previous.
?[searchterm] Backward search.
*You can use regular expressions and escape chars.
Can be used as motions with delete, change, copy.
/ can be replaced by other separators, % # etc.*

Marks

Marks allow you to jump to designated points in your code. m{a-z} Set mark a-z at cursor position
m{A-Z} Sets a global mark (across files)
'{a-z} Goto start of the line of the mark
' ' Go back to the previous jump location

Macros and Registers

q<register><commands>q Record a macro in register
qr Start recording a macro in register "r" (q quits).
@r Execute macro in register "r".
@@ Execute previous macro.

:reg List all registers.
"r Access content of register r.
"ry Yank to register r.
"" Unnamed register has the content of the last yank or
deleted text.
"0 to "9 Previous yanked of deleted text
". The last inserted text
": The last command executed (@: runs again)
"+ and "*" Have the systems clipboard
<C-r><register> On insert mode: paste value of reg

Miscellaneous

:se ft=python sets filetype.
:so ~/file rebuilds a file.
:so % rebuilds this file.
:sav ~/file Saves file elsewhere.
:!ls -lah Access the command line.
:r !date Append command output to cursor.

:%!jq . --tab Prettify JSON file with tabs using jq.

Documentation

<C-] Follow documentation hyperlink.
<C-o> Go back.

Folds

zc Close Fold.
zo Open Fold.
za Toogle Fold.

Equivalents

c	c\$
s	cl
S	^c
I	^i
A	\$a
o	A<CR>
O	ko

Tmux

Sessions

default prefix changed to <C-a>
tmux starts a new session.
tmux new -s NAME starts it with that name.
tmux ls lists the current sessions.
<C-a> d detaches the current session.
tmux a attach to last session.

Windows

<C-a> c Creates a new window. To close it you can just
terminate the shells doing <C-d>.
<C-a> N Go to the N th window. Note they are
numbered.
<C-a> p Goes to the previous window.
<C-a> n Goes to the next window.
<C-a> , Rename the current window.
<C-a> w List current windows.

Panes

<C-a> | Split the current pane horizontally. Default ".
<C-a> - Split the current pane vertically. Default "%.
<M-direction> Move to the pane to direction.
<C-a> z Toggle zoom for the current pane.
<C-a> [Start scrollbar. You can then press <space>
to start a selection and <enter> to copy that selection.
<C-a> <space> Cycle through pane arrangements.

Multiple Monitors

Create sessions for multiple monitors:
tmux new -s left_monitor
tmux new -r left_monitor -s right_monitor

Next item